IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 24 January 2025, accepted 12 March 2025, date of publication 17 March 2025, date of current version 1 April 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3552042

APPLIED RESEARCH

Applying Machine Learning on Big Data With Apache Spark

ELIAS DRITSAS AND MARIA TRIGKA
Department of Informatics and Computer Engineering, University of West Attica, Egaleo Park Campus, 12243 Athens, Greece
Corresponding author: Elias Dritsas (idritsas@uniwa.gr)

The publication of the article in OA mode was financially supported by HEAL-Link.

ABSTRACT The exponential growth of data in the digital age has necessitated the development of frameworks capable of efficiently handling and processing vast datasets. This paper explores the application of machine learning (ML) models within the Apache Spark ecosystem, focusing on the performance and scalability of these models in big data environments. Through comprehensive experiments on three diverse datasets, namely NYC Taxi Trip Duration, Netflix Prize, and Higgs Boson, we address both regression and classification tasks. For the regression tasks using the NYC Taxi Trip Duration and Netflix Prize datasets, we evaluated models including Linear Regression (LinR), Random Forest (RF), Gradient-Boosted Trees (GBT), Support Vector Regressor (SVR), and K-Nearest Neighbors (KNN). For the classification task using the Higgs Boson dataset, we assessed models such as Logistic Regression (LR), RF, GBT, Support Vector Machines (SVM), and KNN. The study employed key performance metrics such as Root Mean Squared Error (RMSE) and Mean Absolute Error (MAE) for regression and Accuracy, Precision, Recall, F1-Score, and Area Under the Curve (AUC) for classification. Our findings indicate that Apache Spark's in-memory processing and distributed computing capabilities provide effective scalability, allowing these models to handle large-scale data with linear increases in processing time. Finally, that study highlights the importance of model selection and resource optimization in big data contexts and provides valuable insights into the practical integration of ML models within the Spark framework.

INDEX TERMS Big data, machine learning, apache spark, data analysis.

I. INTRODUCTION

The advent of the digital age has led to an unprecedented explosion in data generation. With the rise of social media, Internet of Things (IoT) devices, e-commerce, and various other digital platforms, data is being produced at a remarkable rate, leading to the era of big data. Big data is characterized by its volume, velocity, and variety, posing significant challenges for traditional data processing techniques. The need to efficiently analyze and extract meaningful insights from these massive datasets has become increasingly critical for organizations aiming to gain a competitive edge [1], [2].

ML, a subset of artificial intelligence (AI), has emerged as a powerful tool for making sense of big data. By leveraging algorithms that can learn from data, ML enables the discovery of patterns and relationships that are not immediately apparent through traditional analytical methods. The application of ML in big data environments allows for predictive analytics, automated decision-making, and enhanced data-driven strategies across various domains such as finance, healthcare, transportation, and entertainment [3], [4].

However, the integration of ML models with big data comes with its own set of challenges. Traditional data processing frameworks often struggle to scale with the growing size and complexity of datasets. Moreover, ML algorithms typically require substantial computational resources, which can further exacerbate the problem when dealing with big data [5], [6].

Apache Spark, an open-source unified analytics engine, has gained prominence as a robust solution for big data processing. Unlike traditional disk-based processing frame-

The associate editor coordinating the review of this manuscript and approving it for publication was Zhe Xiao.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

works, Spark leverages in-memory computing to perform data processing tasks at high speeds. Its ability to distribute data and computation across a cluster of machines makes it an ideal platform for flexible and efficient big data analytics [7], [8].

Spark's MLlib library offers a range of scalable and highly optimized ML algorithms, making it a compelling choice for big data applications. It provides built-in support for distributed model training and iterative processing, eliminating the need for external tools and ensuring seamless integration with large-scale data pipelines. Leveraging in-memory computing, Spark minimizes data retrieval latency, enhancing the efficiency of iterative ML processes such as gradient-based optimization. Its fault-tolerant architecture further ensures robust execution across dynamic cloud and cluster environments, solidifying Spark as a powerful and scalable framework for ML-driven big data analytics. By combining the strengths of Spark's distributed computing capabilities with advanced ML techniques, significant improvements in processing times and model performance can be achieved [9], [10].

A. MOTIVATION

The unprecedented explosion of data generation across diverse domains in the digital age necessitates the development of scalable and efficient analytics frameworks. While traditional data processing systems struggle to cope with the complexity, volume, and velocity of modern datasets, this research focuses on leveraging Apache Spark's in-memory distributed computing capabilities to integrate advanced ML techniques. The imperative drives the study to evaluate the performance, scalability, and computational efficiency of these models across various high-impact datasets, offering insights into their practical applicability for addressing the critical challenges in big data environments. This work aspires to bridge the gap between theoretical model development and their real-world deployment, providing a roadmap for optimizing decision-making processes in resource-intensive settings.

B. CONTRIBUTION

The contribution of this article is outlined as follows:

- Provides a thorough evaluation of several prominent ML models— LinR, LR, RF, GBT, SVM, SVR, and KNN — within the Apache Spark ecosystem. This evaluation is performed across three diverse datasets—NYC Taxi Trip Duration, Netflix Prize, and Higgs Boson—covering both regression and classification tasks. By leveraging Spark's distributed computing capabilities, we provide insights into the predictive performance, scalability, and applicability of these models in big data contexts.
- Demonstrates the scalability of these ML models by analyzing their training and prediction times on varying dataset sizes, highlighting Apache Spark's capabilities in handling large-scale data efficiently.
- Delves into the computational resource demands of each model, monitoring central processing unit (CPU) and memory usage during both the training and prediction phases. Also, we provide a detailed assessment of resource efficiency, offering a practical insight for optimizing ML workflows in big data environments. Our findings emphasize the trade-offs between computational cost and predictive performance, particularly in resource-constrained settings.

This research article is organized as follows: Section II provides a review of related works relevant to the topic. Section III details the methodology employed in the study. Section IV showcases the experimental evaluation and results. Finally, Section V summarizes our research findings and explores possible future directions.

II. RELATED WORKS

The application of ML models to big data has been an area of significant research interest in recent years. Numerous studies have explored the integration of ML algorithms with big data frameworks to enhance processing efficiency and model performance. This section outlines the existing literature relevant to the intersection of ML, big data, and the utilization of Apache Spark.

Firstly, [11] aims to address the issue of herbicide resistance in agricultural weeds using advanced ML techniques. It reviews various ML approaches and explores their potential in crop protection, specifically through a case study modelling herbicide resistance in annual ryegrass using Markov random fields (MRF). The prevailing technique discussed is the auto binomial model, which incorporates spatial autocorrelation among neighbouring shires to enhance predictive accuracy. The study demonstrates the model's potential by evaluating its performance against traditional LR models, highlighting its practical applicability in agriculture. Similarly, [12] explains the application of ML techniques in the context of big data. The article highlights the societal impacts of ML across various fields, discusses the challenges and opportunities that big data presents for ML, and provides an overview of prominent tools and platforms used for big data analytics, such as Apache Hadoop and Spark. The article emphasizes the necessity of adapting existing ML algorithms to handle the volume, velocity, variety, veracity, and value of big data effectively.

Moreover, [13] compares the performance of two ML frameworks, Apache Spark's MLlib and Apache Hadoop's Mahout, for big data processing. The study focuses on implementing and evaluating three types of ML algorithms: clustering (K-means), classification (LR), and collaborative filtering (Alternating Least Squares). Experimental simulations are conducted to assess the execution time and stability of both frameworks as data size increases. The prevailing model used for comparison is numerically analyzing the running times of the algorithms under different data volumes, highlighting the strengths and limitations of each framework. Also, [14] develops a real-time remote health status

prediction system. The prevailing model used is a decision tree (DT), implemented through Apache Spark's MLlib for ML and Spark Streaming for handling streaming data. That system processes tweets containing health attributes, predicts the user's health status, and sends direct messages to users based on these predictions. The study demonstrates the deployment and effectiveness of this scalable system in both local and cloud environments.

Besides, [15] presents a scalable framework for analyzing mobile big data (MBD) using deep learning (DL) techniques in conjunction with Apache Spark. The framework is designed to handle the high volume, velocity, and variety of MBD by leveraging Spark's distributed computing capabilities to accelerate the learning process of deep models. The authors validate this framework through a context-aware activity recognition system, demonstrating significant improvements in both accuracy and processing speed compared to conventional methods. The main technique emphasized is the iterative MapReduce-based distributed DL implemented on Spark. In the same direction, [16] explores and evaluates the performance of big data analytics using Apache Spark and Hadoop Distributed File System (HDFS) with a focus on Scala and Java programming languages. It compares the performance of these languages using supervised and unsupervised ML algorithms on large datasets. This work reveals that Scala generally performs better than Java, with improvements of 10%-20%, depending on the algorithm.

The purpose of article [17] is to investigate the effectiveness of Apache Spark MLlib 2.0 in big data ML applications. The prevailing technique highlighted is the use of Apache Spark MLlib, a platform known for its high performance and efficiency in handling massive datasets. The study conducts a comparative analysis of Spark MLlib 2.0 and the Weka library across various ML tasks such as classification and clustering, utilizing different large datasets. The authors aim to bridge the gap between big data analysis and ML by examining both qualitative and quantitative attributes of Spark MLlib through large-scale real-world experiments. The findings suggest that Spark MLlib offers superior performance and speed compared to traditional methods like Weka. In like manner, [18] compares the real-time data processing capabilities of two prominent frameworks such as Hadoop MapReduce and Apache Spark. It aims to address the limitations of Hadoop MapReduce in handling real-time data by presenting experimental evaluations using Twitter data. The article highlights the numerical models and techniques of both frameworks, with a specific focus on the architecture and performance comparison. The prevailing technique emphasized is Apache Spark due to its in-memory processing capabilities, which are shown to be more efficient than Hadoop MapReduce in real-time data analysis. In the same direction, [19] explores Apache Spark MLlib as a scalable, distributed ML library for handling large datasets, comparing it against Weka in terms of computational efficiency and accuracy in classification and clustering. The experiments with datasets containing millions of records revealed that Apache Spark MLlib significantly reduces processing time compared to Weka while maintaining comparable accuracy across models like SVM, RF, and Naïve Bayes. These contributions highlight the integration of big data technologies with ML for efficient large-scale analytics, particularly in real-time event detection and classification.

Furthermore, [20] proposes a novel classification method using the Rider Chaotic Biography Optimization algorithm integrated with a deep stacked autoencoder within the Spark framework, demonstrating improved accuracy (86.71%), dice coefficient (92.7%), sensitivity (75.2%), and specificity (95.4%) on large-scale datasets from the UCI repository. The authors in [21] evaluate tree-based machine learning techniques for big data analytics, comparing DT, RF, and GBT within Apache Spark, showing significant performance gains over traditional MapReduce, particularly in reduced read/write times (Spark: 17.23 MB/s read, 19.41 MB/s write vs. MapReduce: 28.65 MB/s read, 35.43 MB/s write) and execution efficiency (Spark: 9.47s vs. MapReduce: 11.57s). Also, [22] focuses on detecting road traffic events from Arabic tweets using machine learning models such as Naïve Bayes, SVM, and LR, integrated with Apache Spark for big data processing. The study demonstrated that their method successfully detects events like accidents, roadwork, and traffic conditions with classification evaluation metrics, achieving high recall and accuracy rates. Notably, the results showed that SVM outperformed the other classifiers in accuracy and F1–score, while LR performed better for certain event types.

Finally, [23] explores sentiment analysis on Twitter data using Apache Spark. The authors propose three classification algorithms: Naive Bayes, LR, and DTs, implemented in Spark to handle large-scale data efficiently. This research is inspired by the graph-based hashtag sentiment classification model from prior work and aims to improve sentiment polarity analysis. The suitability of Apache Spark for big data processing is emphasized, comparing it favourably against MapReduce and Hadoop. Similarly, [24] proposes a methodology for forecasting tourism demand in Greece using data mining and ML techniques. This methodology leverages Apache Spark, a robust analytics engine, to build a multivariate predicting model for tourism demand. The model uses DTs to predict quarterly tourist arrivals from a dataset spanning 2006 to 2015. This approach aims to provide accurate forecasts that can inform decision-making and potentially be adapted for other countries. The prevailing technique is the application of DTs within the Apache Spark environment for scalable and efficient data processing and prediction.

The present research work differentiates itself from prior related studies by providing a comprehensive, multi-dimensional evaluation of ML models within Apache Spark across multiple datasets, covering both regression and classification tasks. Our study empirically validated previously reported trends in Spark-based ML while also

extending prior research through a detailed comparative analysis of computational efficiency and model performance trade-offs in large-scale distributed environments. Unlike previous studies focusing on isolated frameworks or specific algorithms, this work integrates resource utilization metrics (CPU and memory usage) alongside scalability and predictive performance analysis, offering a more holistic assessment. Furthermore, while prior studies, such as those comparing Spark MLlib with Hadoop MapReduce, focused on execution time improvements, this study analyzes computational trade-offs, assesses training and prediction times, and evaluates computational cost versus predictive performance. Additionally, previous research investigating Spark's efficiency in real-time applications (e.g., health status prediction systems) mainly emphasized single-domain applications. In contrast, this work applies its analysis across diverse datasets, including NYC Taxi Trip Duration, Netflix Prize, and Higgs Boson. The study's contribution lies in its emphasis on real-world applicability by benchmarking multiple ML models with detailed performance metrics, bridging critical gaps in the existing literature, and enhancing the practical understanding of ML integration in significant data ecosystems.

III. MATERIALS AND METHODS

This section outlines the comprehensive workflow employed to evaluate the performance of ML models on big data using Apache Spark. The workflow is divided into key phases: data acquisition, preprocessing, model implementation, hyperparameter tuning, model evaluation, and resource utilization monitoring.

A. DATA ACQUISITION AND PREPROCESSING

The initial phase of our methodology involves the systematic acquisition and meticulous preprocessing of data. This phase is crucial as it lays the foundation for all subsequent steps in the workflow, ensuring that the data is clean, relevant, and well-prepared for ML.

1) DATA SOURCE

We began by acquiring three diverse and comprehensive datasets to represent a broad range of domains and analytical challenges. The NYC Taxi Trip Duration dataset includes detailed records of taxi trips in New York city, providing information on pickup and dropoff locations, trip duration, passenger count, distance travelled, and timestamps, among other features. This dataset is particularly useful for time-series analysis and regression tasks. The dataset comprises approximately 100 million trip records, offering a substantial volume of data for analysis and model training. The target variable is trip duration [25].

The Netflix Prize dataset consists of user ratings (target variable) for a vast array of movies. It includes user IDs, movie IDs, ratings (from 1 to 5 stars), and timestamps, providing a source of information for collaborative filtering and recommendation system development. The dataset contains 100 million ratings given by 480,000 users for 17,000 movies, collected between 1998 and 2005 [26].

The Higgs Boson dataset, sourced from high-energy physics experiments, contains data used for binary classification tasks aimed at detecting the presence of the Higgs Boson particle. It includes numerous features derived from particle collisions, which are instrumental for classification and anomaly detection tasks. This dataset comprises approximately 11 million events (instances) [27].

2) DATA PREPROCESSING

Datasets were subjected to a rigorous preprocessing phase to ensure data quality and enhance the efficacy of ML models. This phase included some common but key steps. Several data transformation steps were applied to optimize the dataset for ML model training within Apache Spark's distributed framework. First, categorical and numerical columns were converted to their appropriate data types (integer, float, boolean, timestamp) to ensure seamless compatibility with Spark MLlib. Categorical variables were encoded using one-hot encoding, transforming them into numerical representations suitable for machine learning models. To enhance model performance, numerical features were standardized using Z-score scaling, which normalizes feature distributions and improves the convergence of gradient-based algorithms. These preprocessing steps ensured that the dataset was well-structured, scalable, and optimized for efficient ML model training within Spark's distributed environment.

To evaluate the models effectively, each dataset was divided into training and testing sets. Typically, an 80-20 split was used, where 80% of the data was used for training the models, and the remaining 20% was reserved for testing their performance on unseen data. This step is essential for assessing the generalization ability of the models [28].

The class imbalance is particularly relevant for the Higgs Boson dataset, where the positive class (35% presence of the Higgs Boson) is significantly outnumbered by the negative class (65%). So, the class imbalance was handled by building ensemble methods like RF and GBT which are robust in such cases and measuring evaluation metrics that are insensitive to class imbalance (e.g., AUC-ROC values). In Algorithm 1, we summarize the workflow including training, tuning and evaluation steps.

Through the preprocessing phase, the datasets were transformed into a format, ready for the implementation of ML models. Each preprocessing step was designed to address specific challenges presented by the datasets, ensuring that the data fed into the models was of the highest quality and relevance. This phase is critical in setting the stage for the successful application of ML techniques and achieving robust and reliable results.

B. MODEL IMPLEMENTATION

We implemented several classical ML models using Apache Spark's MLlib library [29], each selected for both classifica-

Algorithm 1 Training, Tuning, and Evaluation Procedure

Input: Original Dataset, k
Output: Final Model Performance on Test Set
procedure Initial Data Splitting
    Split the original dataset into:
        Training Set: 80%
        Test Set: 20%
end procedure
procedure Cross-Validation and Hyperparameter Tuning
    Perform k-fold Cross-Validation on the Training Set:
    for each fold $i = 1$ to k **do**
        Train the model on the $k - 1$ folds (training portion)
        Validate the model on the remaining 1 fold (validation portion)
    end for
    Perform Grid Search over predefined hyperparameters
    Select optimal hyperparameters based on average performance across folds.
end procedure
procedure Final Model Training and Evaluation
    Combine the Training and Validation Sets
    Retrain the model on the combined sets using optimal hyperparameters
    Evaluate the final model on the Test Set
end procedure

tion and regression except for LinR which was used only for regression), due to its simplicity, relevance and applicability to big data analytics.

- The LinR [30] model attempts to establish a linear relationship between the input features (independent variables) and the dependent variable (e.g., predicted movie rating or taxi trip duration). The model assumes that the relationship between the input features (e.g., user ID, movie ID, and timestamp) and the target variable y (movie rating or taxi rip duration) can be expressed as a weighted sum of the input features plus a bias term b:

$$\hat{y} = \sum_{i=1}^{N} w_i x_i + b,$$

  where $x_i, w_i, i = 1, 2, \ldots, n$ are the input features and the weights assigned to each feature, and y is the dependent variable predicted. The parameters of the model are optimized to minimize the difference between the predicted and actual values of the variable y, typically using the MSE as the loss function.
- RF [31] is an ensemble ML model that constructs multiple DTs and merges their results. Each tree T_i in the forest is trained on a bootstrap sample of the data and outputs a class prediction or the target variable predicted value. The final prediction is made by majority voting for classification tasks or averaging for regression tasks. The decision function for a forest of N trees can be expressed as:

$$\hat{y} = \frac{1}{N} \sum_{i=1}^{N} T_i(\mathbf{x}).$$

  RF helps to mitigate overfitting and improve predictive performance by averaging out the errors of individual trees.
- GBT [32] is an ensemble technique that builds models in a stage-wise fashion to optimize a differentiable loss function. The GBT algorithm iteratively adds trees to the ensemble, where each new tree $h_m(\mathbf{x})$ is trained to correct the residual errors of the combined ensemble of previous trees. The model at stage m is given by:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta h_m(\mathbf{x}),$$

  where F_{m-1} is the ensemble model from the previous stage, η is the learning rate, and $h_m(\mathbf{x})$ is the new tree added at stage m. The trees are trained to minimize a loss function $L(y, F_m(\mathbf{x}))$, such as mean squared error for regression or log loss for classification.
- SVM [33] is a supervised ML model that finds the optimal hyperplane to classify data points. For linearly separable data, SVM solves the optimization problem:

$$\min_{\mathbf{w},b} \frac{1}{2} \|\mathbf{w}\|^2$$

  subject to the constraints $y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1$ for all training samples $(\mathbf{x}_i, y_i)$. For non-linearly separable data, SVM uses kernel functions $\phi(\mathbf{x})$ to map input features into a higher-dimensional space, where a linear separator can be found. The decision function for a kernel SVM is:

$$f(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b,$$

  where α_i are the support vector coefficients and $K(\mathbf{x}_i, \mathbf{x}) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x})$ is the kernel function.
- SVR [34] extends the principles of SVM to regression tasks, such as predicting movie ratings in the Netflix Prize dataset or trip durations in the NYC Taxi Trip Duration dataset. Instead of finding a hyperplane that separates classes, SVR finds a function that approximates the target values within a specified margin of tolerance (ϵ). The goal is to minimize the prediction error while maintaining a model that is as flat as possible. It can handle both linear and non-linear regression tasks by utilizing different kernel functions, similar to SVM. This flexibility makes SVR a powerful tool for capturing complex relationships between features and target variables in regression tasks.
- KNN [35] is a non-parametric method used for classification and regression based on the K closest training examples in the feature space. For classification, the

predicted class $\hat{y}$ of a query point $\mathbf{x}$ is determined by majority voting among its K nearest neighbours:

$$\hat{y} = \text{mode}(y_{i_1}, y_{i_2}, \ldots, y_{i_k}),$$

where y_{i_j} are the labels of the k nearest neighbors. For regression, the prediction is the average of the neighbours' values:

$$\hat{y} = \frac{1}{k} \sum_{j=1}^{k} y_{i_j}.$$

KNN's simplicity makes it a useful baseline model, but its performance can degrade with high-dimensional data. Finally, training refers to data preprocessing and structure initialization.

- LR [36] is a linear model used for binary and multi-class classification. Mathematically, it models the probability $P(y = 1|x)$ as a sigmoid function of a linear combination of input features:

$$P(y = 1|x) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}},$$

where $\mathbf{w}$ is the vector of weights, b is the bias term, and $\sigma(z)$ is the sigmoid function. The model parameters are estimated by maximizing the likelihood of the observed data, typically using gradient descent or similar optimization techniques.

C. HYPERPARAMETER TUNING, MODEL TRAINING AND EVALUATION

To optimize model performance, hyperparameter tuning is conducted using grid search and cross-validation. Grid search involves systematically evaluating combinations of hyperparameters by training models on each combination and selecting the best-performing set. Cross-validation involves splitting the training data into $k = 5$ subsets and training the model $k = 5$ times, each time using a different subset as the validation set and the remaining data for training. This process helps to ensure that the model generalizes well to unseen data [37].

Each model is then trained on the training portion (80%) of the dataset using the tuned hyperparameters. The optimal hyperparameters (as shown in Tables 1, 2) for each model are determined based on their performance during 5-fold cross-validation, aiming to maximize key advanced metrics: RMSE, and MAE for the regression tasks [38] (NYC Taxi Trip Duration and Netflix Prize datasets) and accuracy, precision, recall, F1-score, and AUC [39] for the classification task (Higgs Boson Dataset).

The performance of the optimally tuned trained models was evaluated on the testing datasets using the aforementioned metrics defined and calculated as follows:

a: REGRESSION METRICS FOR THE NYC TAXI TRIP DURATION AND NETFLIX PRIZE

The Mean Absolute Error is defined as the average of the absolute differences between the predicted values $\hat{y}_i$ and the actual values y_i over all test samples n.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} \left| y_i - \hat{y}_i \right|.$$

The Root Mean Squared Error is defined as the square root of the average of the squared differences between the predicted values $\hat{y}_i$ and the actual values y_i overall test samples n.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^{n} \left(y_i - \hat{y}_i \right)^2}.$$

TABLE 1. Optimal hyperparameters for the NYC Taxi Trip Duration and Netflix Prize datasets.

Model	NYC Taxi Dataset	Netflix Dataset
LinR	Regularization: 0.1	Regularization: 0.1
RF	N_Estimators: 200 Max_Depth: 10 Min_Samples_Split: 5	N_Estimators: 150 Max_Depth: 12 Min_Samples_Split: 4
GBT	N_Estimators: 300 Learning_Rate: 0.05 Max_Depth: 5	N_Estimators: 250 Learning_Rate: 0.03 Max_Depth: 6
SVR	C: 10 Epsilon: 0.2 Kernel: Linear	C: 15 Epsilon: 0.01 Kernel: Linear
KNN	N_Neighbors: 10	N_Neighbors: 8

TABLE 2. Optimal parameters for the Higgs Boson dataset.

Model	Optimal Parameters
LR	Regularization parameter (λ): 0.01
RF	N_Trees: 100, Max_Depth: 20, Feature subset size: $\sqrt{(N_Features)}$
GBT	Number of iterations: 200, Learning rate: 0.1, Max_Depth: 10
SVM	Regularization parameter (C): 1.0, Type: linear, Max_Iterations: 1000
KNN	N_Neighbours (K): 5, Distance metric: Euclidean

b: CLASSIFICATION METRICS FOR THE HIGGS BOSON DATASET

Accuracy is the proportion of correctly classified instances out of the total instances. Mathematically, accuracy is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN},$$

where TP is the number of true positives, TN is the number of true negatives, FP is the number of false positives, and FN is the number of false negatives.

Precision indicates the proportion of true positive predictions among all positive predictions. Precision is defined as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall also known as sensitivity or true positive rate, recall measures the proportion of true positive predictions among all actual positive instances. Recall is defined as:

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-score is the harmonic mean of precision and recall, providing a balance between the two. It is particularly useful

when the class distribution is imbalanced. The F1-score is defined as:

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

AUC metric represents the area under the receiver operating characteristic (ROC) curve, which plots the true positive rate against the false positive rate at various threshold settings. AUC is defined as:

$$\text{AUC} = \int_0^1 \text{TPR}(\text{FPR}^{-1}(x))\, dx$$

where *TPR* is the true positive rate and *FPR* is the false positive rate.

D. RESOURCE UTILIZATION MONITORING

To understand the computational efficiency of each model, the CPU and memory usage are monitored during both the training and prediction phases. CPU and memory usage are measured as the percentage of CPU and memory resources utilized during model training and prediction. Memory usage is tracked as the amount of Random-access memory (RAM) consumed during these two phases. The following mathematical equations are used to quantify resource utilization.

The CPU utilization U_{CPU} is defined as the ratio $U_{CPU} = \frac{C_{used}}{C_{total}} \times 100$, where C_{used} is the CPU time consumed by the process, and C_{total} is the total available CPU time.

Memory utilization U_{RAM} is defined as $U_{RAM} = \frac{M_{used}}{M_{total}} \times 100$, where M_{used} is the memory used by the process, and M_{total} is the total available memory.

E. EXPERIMENTAL SETUP

All experiments are conducted on a Spark cluster configured to handle large-scale data processing and ML tasks. The cluster comprises 8 nodes, each equipped with 8 cores and 64GB of RAM, providing substantial computational power and memory capacity to efficiently manage and process big data. The HDFS [40] is utilized for distributed storage of the datasets, ensuring efficient data retrieval and storage across the cluster nodes. The system allowed the cluster to scale the number of executors dynamically based on workload demands, ensuring that the 64 GB of RAM per node is efficiently utilized across varying workloads.

Apache Spark version 3.0.1 is employed, leveraging its robust in-memory processing capabilities and optimized MLlib library for scalable ML implementations. The execution mode is set to YARN [41], which manages resource allocation and job scheduling, ensuring optimal utilization of the cluster's resources. This setup provides a highly scalable and efficient environment for conducting comprehensive experiments on the performance of ML models in big data contexts.

Figure 1 illustrates the end-to-end ML model training and evaluation workflow within Apache Spark's distributed computing framework, utilizing DataFrames for efficient data processing, typically imported as CSV files. The process begins with data ingestion and preprocessing, followed by data partitioning into an 80% training set and a 20% test set. ML models from Apache Spark MLlib are then selected and trained using hyperparameter tuning with grid search and 5-fold cross-validation, executed in parallel across Spark's worker nodes to enhance computational efficiency. The trained models undergo distributed inference, with performance evaluation based on key metrics such as RMSE and MAE for regression, or accuracy, precision, recall, F1-score, and AUC for classification. Additionally, computational analysis considers training time, prediction (inference) time, and resource utilization (CPU and memory) to assess model efficiency and scalability within Spark's fault-tolerant, in-memory computing environment.

IV. RESULTS AND DISCUSSION

This section presents a comprehensive analysis of the experimental results obtained from evaluating several classical ML models namely LinR, LR, RF, GBT, SVM, SVR and KNN on three datasets: NYC Taxi Trip Duration, Netflix Prize, and Higgs Boson. The analysis focuses on scalability, model performance, and resource utilization.

A. SCALABILITY

The training and prediction times for the models across different dataset sizes demonstrate the scalability of each algorithm when processed in a distributed environment provided by Apache Spark. The results, summarized in Tables 3, 4, 5 and picturized in Figure 2 show a clear linear relationship between dataset size and training time, highlighting the effectiveness of Apache Spark in handling the processing of large-scale data.

a: TRAINING TIME

LinR reached the fastest training times across all dataset sizes, making it highly efficient and suitable for applications requiring quick model development. The linear nature of LinR resulted in lower computational overhead allowing it to scale effectively with increasing data volumes. SVR and KNN models had moderate training times, with SVR being slower (higher training time) as the dataset size increased. KNN, while simple, required more computational resources as the data grew due to the need to process all data points during training. The need to store and manage large amounts of training data contributes to the increased computational demands. Despite this, KNN remains useful in specific contexts where its simplicity and effectiveness outweigh the computational cost.

RF and GBT regressors showed a reasonable increase in training times as the dataset size grew. As far as GBT is concerned, it showed the longest training times due to the iterative process of building and optimizing trees stage by stage. Besides, the complexity of constructing multiple decision trees within the ensemble method contributed to longer training times compared to simpler models like LinR.

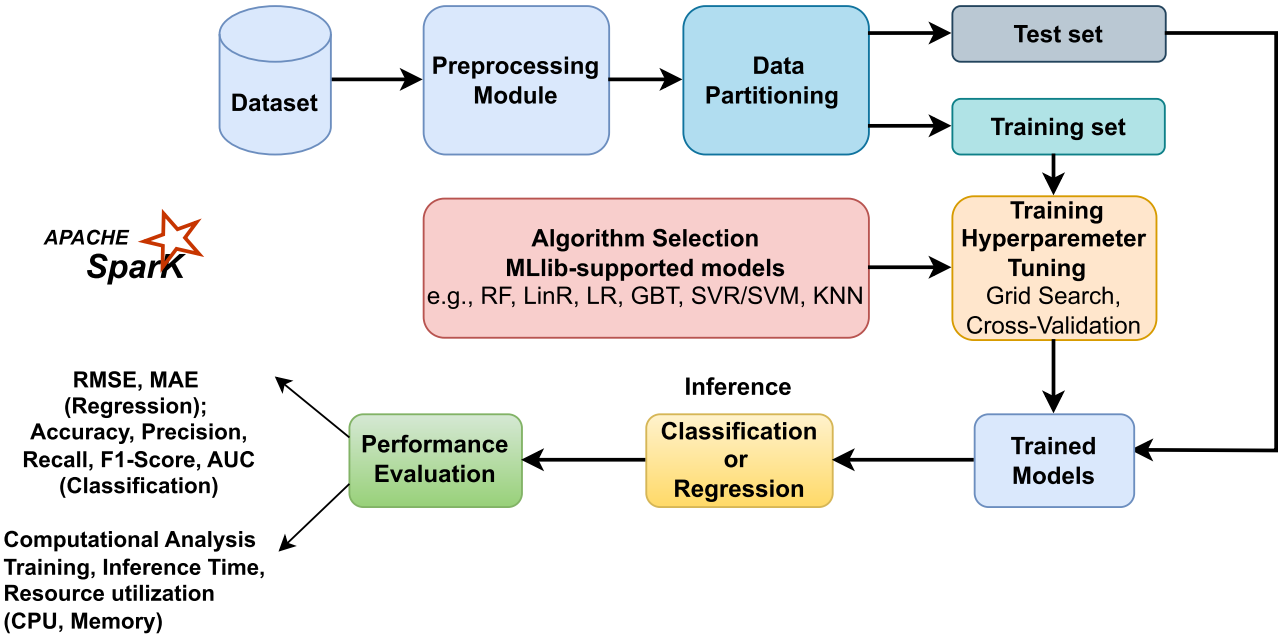


FIGURE 1. ML model training and evaluation workflow in Apache Spark.

TABLE 3. Training and prediction times (minutes) on the NYC Taxi Trip Duration dataset.

% Data	LinR		RF		GBT		SVR		KNN	
	Train	Predict	Train	Predict	Train	Predict	Train	Predict	Train	Predict
10%	4.5	0.5	13.0	1.5	18.5	2.0	5.5	1.0	9.5	1.5
25%	7.0	1.0	23.0	3.0	28.5	4.0	13.0	2.0	18.0	2.5
50%	13.5	2.0	40.0	6.0	47.0	8.0	24.5	4.0	34.0	5.0
75%	19.5	3.0	55.0	9.0	65.0	12.0	38.0	6.0	48.0	7.0
100%	25.5	4.0	70.0	12.0	84.5	16.0	51.5	8.0	61.0	10.0

TABLE 4. Training and prediction times (minutes) on the Netflix Prize dataset.

% Data	LinR		RF		GBT		SVR		KNN	
	Train	Predict	Train	Predict	Train	Predict	Train	Predict	Train	Predict
10%	6.5	0.8	14.0	2.0	18.0	2.5	10.0	1.2	12.0	1.5
25%	9.0	1.5	29.0	4.0	35.0	5.0	21.0	2.5	25.0	3.0
50%	19.0	3.0	53.0	8.0	63.5	10.0	38.0	5.0	44.5	6.0
75%	28.0	4.5	77.0	12.0	90.5	15.0	51.5	7.5	61.5	9.0
100%	37.0	6.0	97.5	16.0	113.5	20.0	65.0	10.0	77.5	12.0

TABLE 5. Training and prediction times (minutes) on the Higgs Boson dataset.

% Data	LR		RF		GBT		SVM		KNN	
	Train	Predict	Train	Predict	Train	Predict	Train	Predict	Train	Predict
10%	8.0	7.0	15.0	9.5	18.0	10.5	10.0	7.5	12.0	9.0
25%	10.5	8.0	35.0	13.0	42.0	14.5	25.0	11.0	29.0	12.0
50%	14.0	10.0	70.0	19.0	85.0	22.0	50.0	14.5	58.0	16.5
75%	21.5	13.5	105.0	25.0	127.0	29.5	75.0	18.5	87.0	21.5
100%	29.0	17.0	140.0	31.0	170.0	37.0	100.0	23.0	116.0	26.5

The increased computational effort required for GBT is offset by its superior performance metrics, making it an ideal choice for scenarios where prediction quality is critical. The model's robustness and high accuracy-reliability justify its use in data-intensive applications with complex datasets, even with the additional computational cost.

Focusing on Table 5 that concerns the classification task with the Higgs Boson dataset, LR exhibited relatively lower training time compared to other models, which was expected due to its simpler nature. As the dataset size increased, both the RF and GBT classifiers exhibited a substantial rise in training time, with the GBT classifier requiring the most

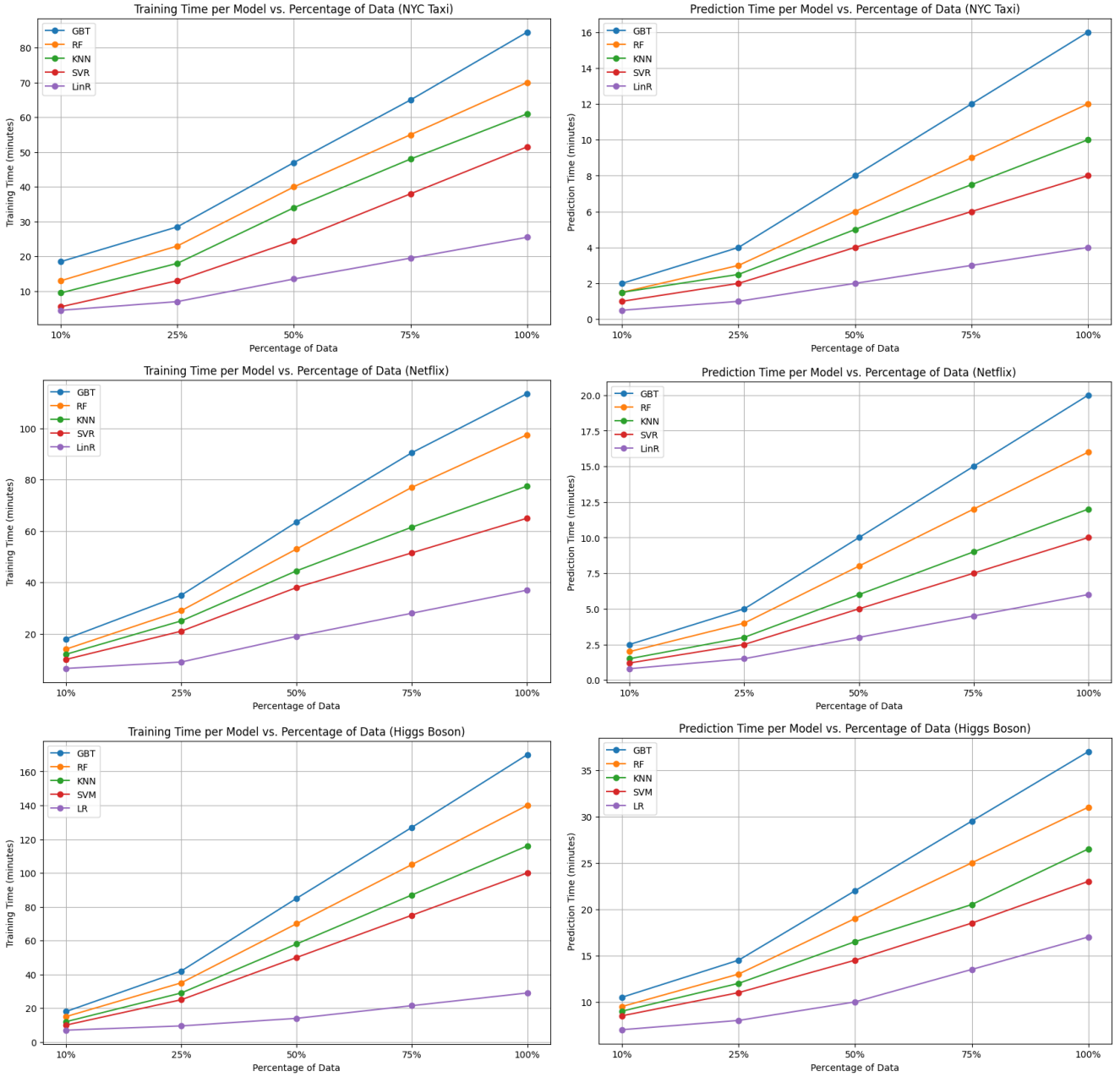


FIGURE 2. **Training (left) and Prediction (right) Time in terms of the data scale for the three datasets.**

time due to the iterative nature of the boosting method. The SVM classifier demonstrated moderate training times, while KNN, consistent across all datasets, showed longer training times than both SVM and LR. This increased duration is attributed to KNN's dependence on processing the entire dataset during training, making it less efficient in handling larger data volumes.

b: PREDICTION TIME

The prediction times for each model across different dataset sizes further highlight their scalability and efficiency in generating predictions once trained. LinR and LR again demonstrated the lowest prediction times, making them highly suitable for real-time applications where quick decision-making is crucial. The simplicity of the model allows for rapid computation, even with large datasets.

SVR and SVM showed prediction times comparable to LinR and LR for low data scales of 10% and 25%, but the gap increased for higher data scales (especially for 75% and 100%) underscoring its efficiency in large-scale classification tasks. SVR, implemented using a linear kernel for scalability, showed increasing prediction times at higher data scales due

to its iterative optimization process. The LinR's and LR's ability to quickly determine the value and class of new data points makes them suitable for applications requiring fast predictions (while maintaining acceptable prediction reliability).

KNN (either as a regressor or classifier) had longer prediction times compared to LinR, and LR, especially with larger datasets. The need to compute distances to all training points for each prediction significantly increases computational demands. This makes KNN less suitable for real-time applications but still valuable for specific use cases where its simplicity and interpretability are desired.

RF (either as a regressor or classifier) provided higher (than LinR and LR) but reasonable prediction times. The ensemble nature of the model, requiring multiple tree evaluations for each prediction, contributes to the increased time. Nevertheless, RF's ability to deliver highly accurate predictions makes it a strong candidate for applications where both speed and accuracy are important.

GBT exhibited even longer prediction times, a consequence of the ensemble method's iterative nature. Unlike RF, where each prediction involves evaluating multiple decision trees independently, GBT adds layers of complexity through its boosting process, where trees are sequentially built to correct the errors of previous ones. This results in a more computationally intensive prediction phase. However, the enhanced accuracy provided by GBT often justifies the additional prediction time, especially in scenarios where the quality of predictions is critical and outweighs the need for rapid processing Overall, Apache Spark's distributed computing capabilities ensure effective scalability and performance for various ML models.

B. MODEL PREDICTION PERFORMANCE

Each ML model's performance was assessed across the three datasets: NYC Taxi Trip Duration, Netflix Prize, and Higgs Boson. Depending on the task under consideration in each dataset, different key metrics were measured: i) RMSE, MAE for the regression task in NYC Taxi Trip Duration and Netflix Prize datasets and ii) accuracy, precision, recall, F1-score, and AUC for the classification task in Higgs Boson dataset. The following analysis provides a detailed discussion of the performance of each model on these datasets.

The NYC Taxi Trip Duration dataset is used to predict the duration of taxi trips in New York City. The performance metrics for each model on this dataset are summarized in Table 6.

Based on Table 6, which evaluates the models using the NYC Taxi Trip Duration dataset, the RMSE and MAE metrics are utilized for understanding their performance. RF and GBT consistently outperform the other models across all data scales, showing the lowest RMSE and MAE values. As the data scale increases from 10% to 100%, RF and GBT maintain their performance advantage, with GBT slightly edging out RF at larger scales, suggesting its robustness in handling complex data interactions. SVR and KNN lag behind RF and GBT, with higher error rates, particularly in larger datasets, indicating their relative inefficiency in capturing the intricacies of the NYC Taxi Trip Duration data. LinR, while computationally efficient, shows the highest RMSE and MAE, making it less suitable for this predictive task where accuracy is paramount.

The Netflix Prize dataset is used for predicting user preferences by estimating movie ratings on a 1-to-5 star scale, which serves as the target variable. The performance metrics for each model on this dataset are summarized below.

Table 7, which evaluates model performance on the Netflix Prize dataset, reveals a similar trend, with RF and GBT leading in predictive accuracy. However, as the dataset scale increases, the performance gap between the two models widens, with GBT consistently achieving the lowest RMSE and MAE values, reaffirming its position as the most accurate model for this task. While RF maintains strong performance, it exhibits slightly higher error metrics than GBT yet remains superior to SVR, KNN, and LinR. This analysis highlights GBT's superiority across both datasets as data complexity and scale increase, while also reinforcing the limitations of simpler models like LinR in such contexts.

The Higgs Boson dataset is used for solving a binary classification task to detect the presence of the Higgs Boson particle. The performance metrics for each model on this dataset are summarized in Table 8 revealing a hierarchy in model effectiveness when evaluated across the key metrics and particularly as the data scale increases. The classification of the Higgs Boson particle is a complex task, demanding models that can accurately distinguish between subtle patterns in the data, which is reflected in the performance of the different models.

GBT and RF emerge as the top performers, with GBT slightly leading. At full data scale, GBT achieves an accuracy of 90.5%, precision of 91.5%, recall of 89.5%, an F1-score of 90.5%, and an AUC of 0.97. This indicates that GBT is exceptionally adept at identifying the Higgs Boson particles, minimizing both false positives and false negatives, which is crucial in scientific tasks where the cost of misclassification can be significant. RF, with an accuracy of 90.0%, precision of 91.0%, recall of 89.2%, F1-score of 90.1%, and an AUC of 0.96, also shows strong performance, particularly in handling complex interactions within the dataset. The ensemble nature of both GBT and RF, which combines multiple decision trees, allows these models to effectively capture the intricate patterns necessary for the Higgs Boson classification.

SVM also perform well but is less effective than GBT and RF. At full data scale, SVM achieves an accuracy of 88.5%, precision of 89.5%, recall of 87.5%, F1-score of 88.5%, and an AUC of 0.95. While SVM is capable of finding the optimal hyperplane to separate classes, it does not match the nuanced pattern recognition that the ensemble methods offer, which is critical in detecting the presence of the Higgs Boson.

TABLE 6. Performance evaluation (in minutes) of ML models on the NYC Taxi Trip Duration dataset.

% Data	LinR		RF		GBT		SVM		KNN	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
10%	9.6	7.3	7.1	5.3	6.8	5.1	8.7	6.8	8.1	6.2
25%	9.2	7.0	6.8	5.1	6.5	4.9	8.4	6.5	7.8	6.0
50%	8.9	6.8	6.5	4.9	6.2	4.7	8.1	6.3	7.5	5.7
75%	8.5	6.5	6.2	4.6	5.9	4.4	7.8	6.0	7.2	5.5
100%	8.1	6.2	5.8	4.3	5.6	4.1	7.5	5.8	6.9	5.2

TABLE 7. Performance evaluation of ML models on the Netflix Prize dataset.

% Data	LinR		RF		GBT		SVM		KNN	
	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE	RMSE	MAE
10%	1.35	1.10	1.20	0.95	1.10	0.88	1.45	1.18	1.55	1.25
25%	1.30	1.05	1.15	0.92	1.05	0.85	1.40	1.15	1.50	1.22
50%	1.25	1.02	1.10	0.90	1.00	0.83	1.35	1.12	1.45	1.18
75%	1.20	0.98	1.08	0.87	0.97	0.81	1.32	1.10	1.42	1.15
100%	1.18	0.95	1.05	0.85	0.95	0.78	1.30	1.08	1.40	1.12

LR, known for its computational efficiency, shows respectable results but falls short in this complex classification task. With an accuracy of 87.5%, precision of 88.5%, recall of 86.8%, F1-score of 87.6%, and an AUC of 0.94 at full data scale, LR demonstrates that while it can handle large datasets, it lacks the depth to fully exploit the complex relationships within the Higgs Boson data, leading to lower overall performance compared to more sophisticated models.

KNN recorded the lowest performance metrics across all measures, particularly as the data scale increased. At 100% data scale, KNN achieved an accuracy of 86.0%, precision of 87.0%, recall of 85.5%, F1-score of 86.2%, and an AUC of 0.93. The high computational demands and the simplistic nature of KNN, which relies on distance calculations, limit its effectiveness in the Higgs Boson classification task where precision is paramount.

For the classification of Higgs Boson particles, GBT and RF are the most suitable models, providing the highest accuracy and robustness across all metrics, making them ideal for this scientifically significant and complex task. SVM, while effective, falls behind the ensemble methods in capturing the intricate patterns necessary for precise classification. LR and KNN, although useful in less complex scenarios, do not meet the demands of this high-precision classification task, as reflected in their lower performance metrics.

Overall, Apache Spark's in-memory processing and distributed computing capabilities significantly enhance the scalability and performance of these ML models on big data.

C. RESOURCE UTILIZATION

Resource utilization analysis provides insights into the computational efficiency of each model, focusing on CPU and memory usage during the training and prediction phases. The following discussion details the resource utilization observed for each model across the different datasets, highlighting the efficiency and scalability of the models within the Apache Spark environment.

1) NYC TAXI TRIP DURATION DATASET

The CPU and memory usage during the training and prediction phases for the NYC Taxi Trip Duration dataset are summarized below (Table 9).

a: TRAINING

LinR demonstrated the lowest CPU usage (up to 50%) and RAM consumption (up to 40%), highlighting its exceptional computational efficiency. This makes LinR particularly well-suited for scenarios where computational resources are constrained and rapid model training is essential.

SVR exhibited moderate CPU and RAM usage, slightly higher than LinR but still lower than RF and GBT. This positions SVR as a balanced option, offering a good compromise between resource efficiency and predictive performance. The KNN regressor showed relatively high resource consumption, with CPU usage reaching up to 65% and RAM usage up to 55% during training. This is primarily due to its reliance on storing and processing large datasets, which limits its scalability and makes it less ideal for very large datasets. RF and GBT regressors displayed a noticeable increase in resource usage. RF consumed up to 70% CPU and 60% RAM, while GBT pushed these figures even higher, reaching 75% CPU and 65% RAM. The resource-intensive nature of GBT, driven by its iterative boosting process, is accompanied by the corresponding improvements in model accuracy.

b: PREDICTION

LinR consistently demonstrates exceptional efficiency, with the lowest resource usage across all models, making it particularly well-suited for real-time predictions where low latency is paramount.

RF and GBT, on the other hand, require significantly more computational resources during the prediction phase, with GBT being the most demanding. However, this higher resource consumption is justified by their enhanced robustness and accuracy, making them valuable in applications where predictive performance is a critical priority.

TABLE 8. Performance evaluation of ML models on the Higgs Boson dataset.

Model	% Data	Accuracy	Precision	Recall	F1-Score	AUC
LR	10%	84.5%	85.0%	83.5%	84.2%	0.90
	25%	85.5%	86.0%	84.8%	85.4%	0.91
	50%	86.5%	87.0%	85.8%	86.4%	0.92
	75%	87.0%	88.0%	86.5%	87.2%	0.93
	100%	87.5%	88.5%	86.8%	87.6%	0.94
RF	10%	87.0%	88.0%	86.5%	87.2%	0.92
	25%	88.0%	89.0%	87.0%	88.0%	0.93
	50%	89.0%	90.0%	88.5%	89.2%	0.94
	75%	89.5%	90.5%	88.8%	89.6%	0.95
	100%	90.0%	91.0%	89.2%	90.1%	0.96
GBT	10%	87.5%	89.0%	86.8%	87.9%	0.93
	25%	88.5%	89.5%	87.5%	88.5%	0.94
	50%	89.5%	90.5%	88.8%	89.6%	0.95
	75%	90.0%	91.0%	89.0%	90.0%	0.96
	100%	90.5%	91.5%	89.5%	90.5%	0.97
SVM	10%	85.5%	86.0%	84.5%	85.2%	0.91
	25%	86.5%	87.0%	85.5%	86.2%	0.92
	50%	87.5%	88.0%	86.5%	87.2%	0.93
	75%	88.0%	89.0%	87.0%	88.0%	0.94
	100%	88.5%	89.5%	87.5%	88.5%	0.95
KNN	10%	83.0%	84.0%	82.5%	83.2%	0.89
	25%	84.0%	85.0%	83.5%	84.2%	0.90
	50%	85.0%	86.0%	84.5%	85.2%	0.91
	75%	85.5%	86.5%	85.0%	85.7%	0.92
	100%	86.0%	87.0%	85.5%	86.2%	0.93

When comparing SVR and KNN, SVR exhibits moderate resource usage, positioning it as a balanced option between efficiency and performance. In contrast, KNN remains the most resource-intensive during prediction, which could hinder its practicality in scenarios where quick, time-sensitive decisions are required. Finally, Figure 3 illustrates the selected ML models' resource consumption, with CPU and memory usage demonstrating a linear scaling with dataset size.

2) NETFLIX PRIZE DATASET

The CPU and memory usage during the training and prediction phases for the Netflix Prize dataset are summarized below (Table 10).

a: TRAINING

LinR once again reached the lowest resource consumption with CPU (up to 56%) and RAM usage (up to 50%) underscoring its efficiency in handling large-scale data with minimal computational overhead. SVR exhibited a more balanced resource profile, while KNN's high resource usage (up to 69% CPU and 60% RAM) during training highlights its limitations in scalability. RF's CPU usage peaks at 73%, with RAM usage reaching 65%, while GBT remains the most resource-intensive, requiring up to 76% CPU and 70% RAM. These models offer high accuracy, justifying their resource demands in complex predictive tasks.

b: PREDICTION

LinR continued to be the most resource-efficient which highlights its suitability for large-scale recommendation systems where quick model development and low resource usage are crucial.

SVR continued to exhibit efficient resource utilization, with resource consumption (slightly) higher than LinR but lower than the ensemble models. This makes SVR a balanced choice for recommendation tasks that require efficient computation and robust performance. KNN's resource usage was the highest (against LinR and SVR), particularly for larger-size data, due to the intensive distance computations required for each prediction making it less suitable for very large datasets despite its effectiveness in specific scenarios.

RF and GBT maintained the highest resource demands, with GBT being the most resource-intensive. Their use is justified in applications where prediction quality outweighs the need for minimal resource usage. RF, while resource-demanding, leveraged its ensemble approach to achieve high estimation accuracy at the cost of increased computational power. The higher CPU and memory usage reflect the model's complexity in building and evaluating multiple trees. GBT maintained the highest resource usage due to its boosting process, requiring significant CPU and memory to optimize and build numerous trees iteratively. This substantial resource investment is justified by the model's superior predictive performance.

From Figure 4, all models showed a resource consumption with CPU and memory usage linearly scaled with dataset size.

3) HIGGS BOSON DATASET

The CPU and memory usage during the training and prediction phases for the Higgs Boson dataset are summarized below (Table 11).

TABLE 9. Resource utilization on the NYC Taxi Trip Duration dataset (regression task).

%Data	LinR		KNN		GBT		SVR		RF	
Training										
	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}
10%	30.0%	6.5%	45.0%	19.0%	55.0%	28.0%	40.0%	14.0%	50.0%	24.0%
25%	35.0%	12.5%	50.0%	25.0%	60.0%	34.5%	45.0%	20.0%	55.0%	30.0%
50%	40.0%	22.0%	55.0%	35.0%	65.0%	45.0%	50.0%	30.0%	60.0%	40.0%
75%	45.0%	31.0%	60.0%	45.0%	70.0%	55.0%	55.0%	39.0%	65.0%	50.0%
100%	50.0%	40.0%	65.0%	55.0%	75.0%	65.0%	60.0%	48.0%	70.0%	60.0%
Prediction										
	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}
10%	24.0%	6.0%	36.0%	12.0%	45.0%	16.0%	32.0%	9.0%	40.0%	16.0%
25%	28.0%	12.0%	40.0%	18.5%	50.0%	22.0%	36.0%	15.0%	45.0%	22.0%
50%	32.0%	18.0%	44.0%	25.0%	54.0%	28.0%	40.0%	21.0%	49.0%	28.0%
75%	36.0%	24.0%	48.0%	32.0%	59.0%	35.0%	44.0%	28.0%	53.0%	35.0%
100%	40.0%	30.0%	52.0%	39.0%	65.0%	42.0%	480.%	35.0%	59.0%	42.0%

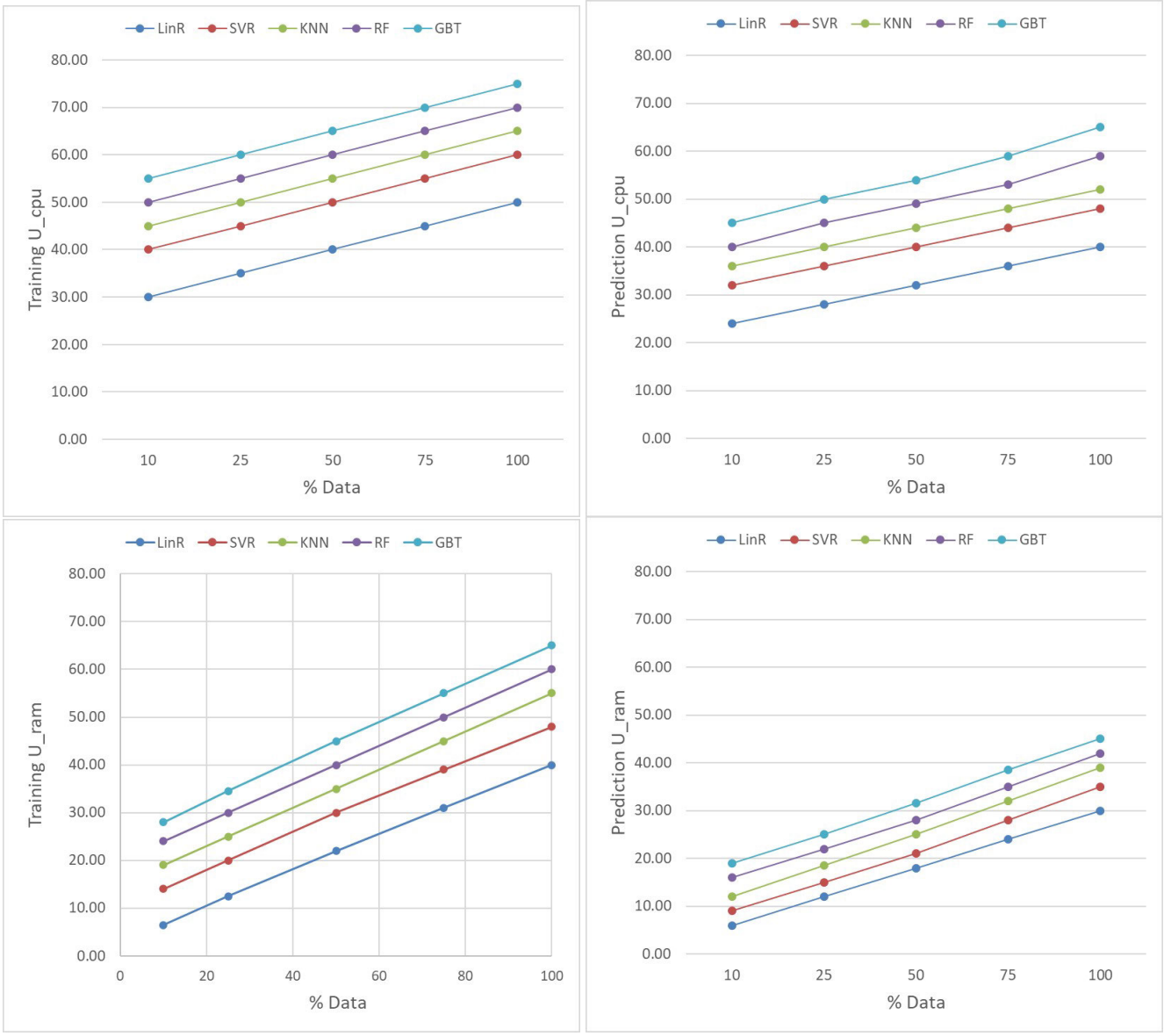


FIGURE 3. Resources utilization during training and prediction in terms of the data scale on the NYC Taxi Trip Duration dataset.

TABLE 10. Resource utilization on the Netflix Prize dataset (regression task).

%Data	LinR		RF		GBT		SVR		KNN	
Training										
	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}
10%	36.0%	15.0%	53.0%	30.0%	57.0%	35.0%	43.0%	20.0%	49.0%	25.0%
25%	41.0%	20.0%	58.0%	35.0%	62.0%	40.0%	48.0%	25.0%	54.0%	30.0%
50%	46.0%	30.0%	63.0%	45.0%	67.0%	50.0%	53.0%	35.0%	59.0%	40.0%
75%	51.0%	40.0%	68.0%	55.0%	71.5%	60.0%	58.0%	45.0%	64.0%	50.0%
100%	56.0%	50.0%	73.0%	65.0%	76.0%	70.0%	63.0%	55.0%	69.0%	60.0%
Prediction										
	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}
10%	25.5%	8.50%	45.0%	20.0%	50.0%	24.0%	34.0%	11.5%	40.0%	15.5%
25%	31.0%	15.0%	50.0%	27.0%	55.0%	31.0%	39.0%	18.0%	45.0%	22.0%
50%	36.0%	22.0%	55.0%	35.0%	60.0%	39.0%	44.0%	25.0%	50.0%	30.0%
75%	41.0%	30.0%	60.0%	43.0%	65.0%	47.0%	49.0%	33.0%	55.0%	38.0%
100%	46.0%	37.0%	65.0%	50.5%	70.0%	55.0%	54.0%	40.0%	60.0%	45.0%

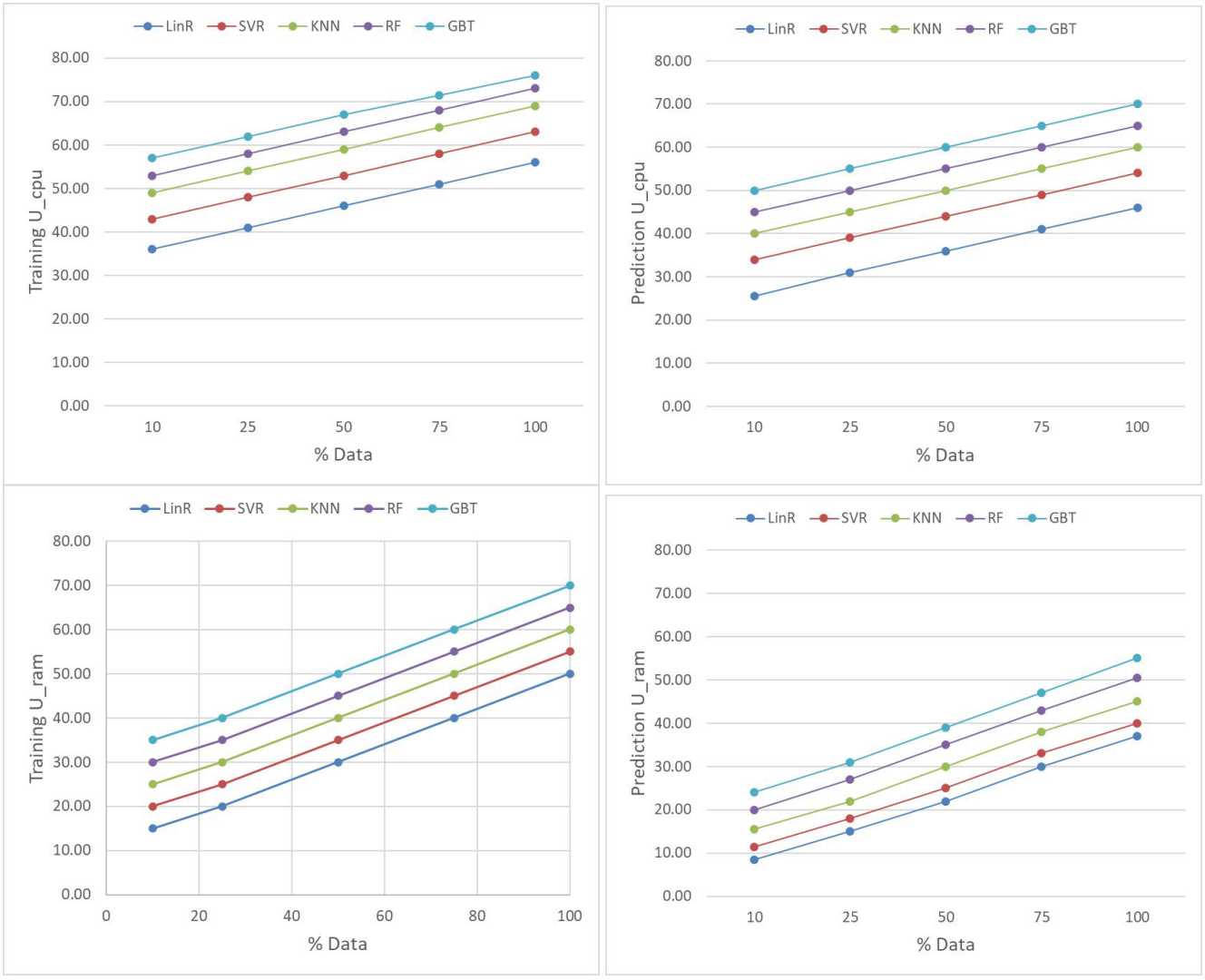


FIGURE 4. Resources utilization during training and prediction in terms of the data scale on the Netflix Prize dataset (regression task).

a: TRAINING

LR classifier demonstrated the lowest resource utilization, making it appropriate for environments where computational efficiency is paramount. SVM exhibited moderate resource usage, making it a good middle-ground choice for large-scale classification tasks. KNN again indicated higher resource usage than SVM, indicating its limited applicability for large datasets. RF and GBT showed significant resource demands, with GBT needing the highest which reflects their complexity due to the iterative nature of GBT.

TABLE 11. Resource utilization on the Higgs Boson dataset (classification task).

%Data	LR		RF		GBT		SVM		KNN	
Training										
	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}
10%	40.0%	18.0%	55.0%	30.0%	60.0%	34.5%	45%	22.0%	50.0%	26.0%
25%	45.0%	23.0%	60.0%	35.0%	65.0%	40.0%	50%	27.0%	55.0%	31.0%
50%	50.0%	31.0%	65.0%	44.0%	70.0%	49.0%	55%	36.0%	60.0%	40.0%
75%	55.0%	39.5%	70.0%	53.0%	75.0%	57.5%	60%	44.0%	65.0%	49.0%
100%	60.0%	48.0%	75.0%	61.0%	80.0%	66.0%	65%	53.0%	70.0%	57.0%
Prediction										
	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}	U_{CPU}	U_{RAM}
10%	35.0%	13.0%	50.0%	25.0%	55.0%	31.0%	40.0%	17.0%	45.0%	20.0%
25%	40.0%	19.5%	55.0%	32.0%	60.0%	38.0%	45.0%	23.5%	50.0%	27.5%
50%	45.0%	26.0%	60.0%	40.0%	65.0%	45.0%	50.0%	31.0%	55.0%	35.0%
75%	50.0%	33.5%	65.0%	48.0%	70.0%	53.0%	55.0%	39.0%	60.0%	43.0%
100%	55.0%	42.0%	70.0%	56.0%	75.0%	61.5%	60.0%	46.5%	65.0%	51.0%

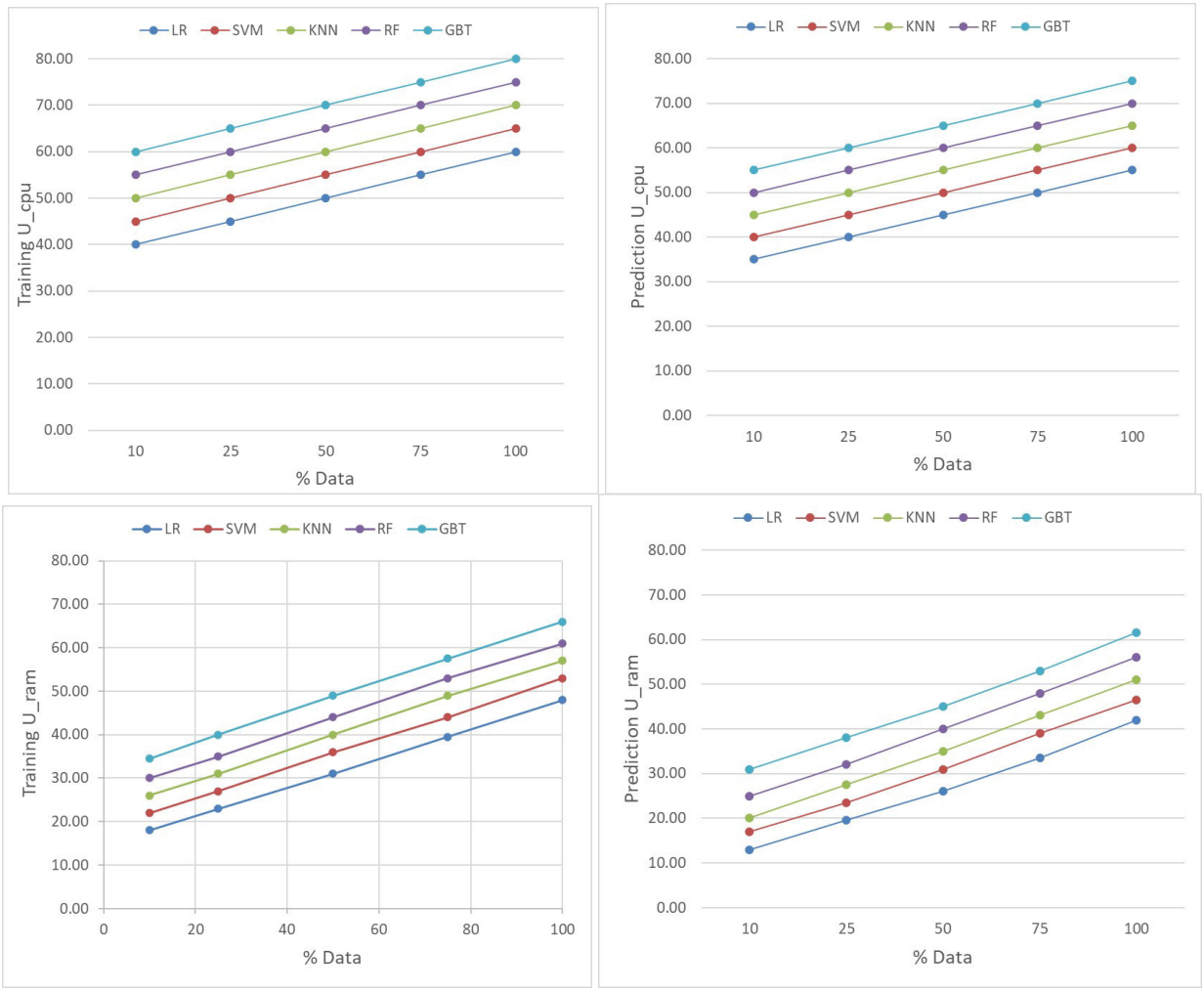


FIGURE 5. Resources utilization during training and prediction in terms of the data scale on the Higgs Boson dataset (classification task).

b: PREDICTION

LR remained the most resource-efficient model - showing the lowest CPU (ranging from 35% to 55%) and memory usage (ranging from 13% to 42%) across all dataset sizes - which is beneficial for real-time classification tasks. Its simplicity and efficiency make it a practical choice for high-energy physics experiments where quick and reliable classification is necessary. SVM maintained moderate resource usage, while KNN's higher demands make it less suitable for real-time applications. KNN's resource usage was substantial, due to

the need to compute distances to all training points for each prediction resulting in high CPU and memory consumption. RF and GBT classifiers continued to demand more resources during prediction, with GBT being the most intensive. RF demonstrated high resource consumption, due to the computational demands of building multiple decision trees and aggregating their results. Both models increased resource usage is balanced by their high classification performance and robustness.

The results are also depicted in Figure 5 showing a linear association between dataset scale and resource utilization, highlighting the effectiveness of Apache Spark in handling large-scale data processing.

D. FINDINGS INSIGHTS

The findings indicate that while tree-based models (RF, GBT) and linear models (LinR, LR) effectively leverage Spark's parallelism, instance-based models (KNN) and iterative optimization models (SVM/SVR) encounter scalability challenges in distributed environments. KNN's reliance on full dataset lookups results in high memory consumption and slow predictions, while SVM/SVR's iterative optimization process demands frequent inter-node communication, leading to computational inefficiencies. These limitations highlight the importance of selecting models that natively support parallelization and batch processing for large-scale Spark-based ML workflows, ensuring both efficiency and scalability.

V. CONCLUSION

This research comprehensively evaluated the application of several ML models namely LinR, LR, RF, GBT, SVM, SVR, and KNN on large-scale datasets using Apache Spark. The datasets used included NYC Taxi Trip Duration, Netflix Prize, and Higgs Boson, each representing different domains and data challenges. Our study focused on key performance metrics, scalability, and resource utilization, providing valuable insights into the efficiency and effectiveness of these models in big data environments.

The experimental results highlighted the significant benefits of using Apache Spark for large-scale ML. Apache Spark's in-memory processing and distributed computing capabilities ensured that all models demonstrated linear scalability, with training and prediction times increasing proportionately with dataset size. Among the models, RF and GBT consistently provided the highest accuracy, precision, recall, F1-score, and AUC across all datasets, making them robust and reliable choices for complex data scenarios. LinR, LR, SVM and SVR offered competitive performance with lower computational costs, making them ideal for applications requiring efficient and quick model deployment. KNN, while effective in certain contexts, showed higher computational demands, which limits its scalability for very large datasets.

Resource utilization analysis revealed that LinR, LR, SVM, and SVR were the most resource-efficient models, with lower CPU and memory usage across all datasets. These models are suitable for environments with limited computational resources, offering a balance between performance and efficiency. RF and GBT, despite their higher resource consumption, provided superior predictive performance, justifying the increased computational costs in scenarios where high accuracy and robustness are critical. KNN's substantial resource usage underscores the need for careful consideration in large-scale applications, although it remains valuable for specific use cases where simplicity and interpretability are prioritized.

Building on this research, future work can explore advanced hyperparameter optimization techniques like Bayesian optimization and genetic algorithms to enhance performance. Furthermore, dimensionality reduction techniques such as Principal Component Analysis (PCA) [42] will be investigated to reduce the number of features in the datasets. This step will help in simplifying the models, reducing overfitting, and decreasing computational costs. More sophisticated ensemble methods and hybrid models could improve predictive accuracy and robustness. Developing real-time ML models within Apache Spark can meet the growing demand for real-time analytics. Optimizing scalability to reduce resource consumption and execution times will benefit large-scale applications. Integrating Apache Spark with other big data technologies like Hadoop, Kafka, and NoSQL databases can enhance data processing and analytics pipelines.

REFERENCES

[1] M. Naeem, T. Jamal, J. Diaz-Martinez, S. A. Butt, N. Montesano, M. I. Tariq, E. De-la Hoz-Franco, and E. De-La-Hoz-Valdiris, "Trends and future perspective challenges in big data," in *Proc. 6th Euro-China Conf. Intell. Data Anal. Appl.*, Arad, Romania. Cham, Switzerland: Springer, 2022, pp. 309–325.

[2] H. B. Alwan and K. R. Ku-Mahamud, "Big data: Definition, characteristics, life cycle, applications, and challenges," *IOP Conf. Ser., Mater. Sci. Eng.*, vol. 769, no. 1, Feb. 2020, Art. no. 012007.

[3] I. K. Nti, J. A. Quarcoo, J. Aning, and G. K. Fosu, "A mini-review of machine learning in big data analytics: Applications, challenges, and prospects," *Big Data Mining Anal.*, vol. 5, no. 2, pp. 81–97, Jun. 2022.

[4] A. E. Hassanien and A. Darwish, *Machine Learning and Big Data Analytics Paradigms: Analysis, Applications and Challenges*. Cham, Switzerland: Springer, 2020.

[5] A. Adadi, "A survey on data-efficient algorithms in big data era," *J. Big Data*, vol. 8, no. 1, p. 24, Jan. 2021.

[6] I. H. Sarker, "Machine learning: Algorithms, real-world applications and research directions," *Social Netw. Comput. Sci.*, vol. 2, no. 3, p. 160, May 2021.

[7] E. Shaikh, I. Mohiuddin, Y. Alufaisan, and I. Nahvi, "Apache spark: A big data processing engine," in *Proc. 2nd IEEE Middle East North Afr. Commun. Conf. (MENACOMM)*, Nov. 2019, pp. 1–6.

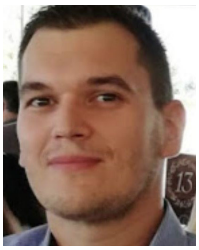
[8] *Apache_Spark*. Accessed: Aug. 12, 2024. [Online]. Available: https://spark.apache.org/

[9] A. H. Ali, M. N. Abbod, M. K. Khaleel, M. A. Mohammed, and T. Sutikno, "Large scale data analysis using MLlib," *TELKOMNIKA-Telecommun. Comput. Electron. Control*, vol. 19, no. 5, pp. 1735–1746, Oct. 2021.

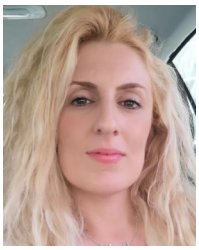
[10] B. Quinto and B. Quinto, "Introduction to spark and spark MLlib," in *Next-Generation Machine Learning With Spark: Covers XGBoost, LightGBM, Spark NLP, Distributed Deep Learning With Keras, and More*. CA, USA: Apress, 2020, pp. 29–96.

[11] R. H. L. Ip, L.-M. Ang, K. P. Seng, J. C. Broster, and J. E. Pratley, "Big data and machine learning for crop protection," *Comput. Electron. Agricult.*, vol. 151, pp. 376–383, Aug. 2018.

[12] K. Divya, P. Bhargavi, and S. Jyothi, "Machine learning algorithms in big data analytics," *Int. J. Comput. Sci. Eng.*, vol. 6, no. 1, pp. 63–70, Jan. 2018.
[13] K. Aziz, D. Zaidouni, and M. Bellafkih, "Big data processing using machine learning algorithms: MLlib and mahout use case," in *Proc. 12th Int. Conf. Intell. Syst., Theories Appl.*, Oct. 2018, pp. 1–6.
[14] L. R. Nair, S. D. Shetty, and S. D. Shetty, "Applying spark based machine learning model on streaming big data for health status prediction," *Comput. Electr. Eng.*, vol. 65, pp. 393–399, Jan. 2018.
[15] M. A. Alsheikh, D. Niyato, S. Lin, H.-P. Tan, and Z. Han, "Mobile big data analytics using deep learning and apache spark," *IEEE Netw.*, vol. 30, no. 3, pp. 22–29, May 2016.
[16] H. K. Omar and A. K. Jumaa, "Big data analysis using apache spark MLlib and Hadoop HDFS with Scala and Java," *Kurdistan J. Appl. Res.*, vol. 4, no. 1, pp. 7–14, May 2019.
[17] Z. Hasan, H. J. Xing, and M. I. Magray, "Big data machine learning using apache spark MLlib," *Mesopotamian J. Big Data*, vol. 2022, pp. 1–11, Jan. 2022.
[18] K. Aziz, D. Zaidouni, and M. Bellafkih, "Real-time data analysis using spark and Hadoop," in *Proc. 4th Int. Conf. Optim. Appl. (ICOA)*, Apr. 2018, pp. 1–6.
[19] M. Assefi, E. Behravesh, G. Liu, and A. P. Tafti, "Big data machine learning using apache spark mllib," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, Feb. 2017, pp. 3492–3498.
[20] A. V. Brahmane and B. C. Krishna, "Big data classification using deep learning and apache spark architecture," *Neural Comput. Appl.*, vol. 33, no. 22, pp. 15253–15266, Nov. 2021.
[21] A. Sheshasaayee and J. Lakshmi, "An insight into tree based machine learning techniques for big data analytics using apache spark," in *Proc. Int. Conf. Intell. Comput., Instrum. Control Techn.*, 2017, pp. 1–12.
[22] E. Alomari, R. Mehmood, and I. Katib, "Road traffic event detection using Twitter data, machine learning, and apache spark," in *Proc. IEEE SmartWorld, Ubiquitous Intell. Comput., Adv. Trusted Comput., Scalable Comput. Commun., Cloud Big Data Comput., Internet People Smart City Innov. (SmartWorld/SCALCOM/UIC/ATC/CBDCom/IOP/SCI)*, Aug. 2019, pp. 1888–1895.
[23] E. Dritsas, I. E. Livieris, K. Giotopoulos, and L. Theodorakopoulos, "An apache spark implementation for graph-based hashtag sentiment classification on Twitter," in *Proc. 22nd Pan-Hellenic Conf. Informat.*, Nov. 2018, pp. 255–260.
[24] N. Ntaliakouras, G. Vonitsanos, A. Kanavos, and E. Dritsas, "An apache spark methodology for forecasting tourism demand in Greece," in *Proc. 10th Int. Conf. Inf., Intell., Syst. Appl. (IISA)*, Jul. 2019, pp. 1–5.
[25] *Apache_Trip*. Accessed: Aug. 12, 2024. [Online]. Available: https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
[26] (2024). *Netflix_Data*. Accessed: Aug. 12, 2024. [Online]. Available: https://www.kaggle.com/datasets/netflix-inc/netflix-prize-data
[27] *Higgs_Boson*. Accessed: Aug. 12, 2024. [Online]. Available: https://archive.ics.uci.edu/dataset/280/higgs
[28] J. J. Salazar, L. Garland, J. Ochoa, and M. J. Pyrcz, "Fair train-test split in machine learning: Mitigating spatial autocorrelation for improved prediction accuracy," *J. Petroleum Sci. Eng.*, vol. 209, Feb. 2022, Art. no. 109885.
[29] *ML_MLib*. Accessed: Aug. 12, 2024. [Online]. Available: https://spark.apache.org/mllib/
[30] A. Lerch, "Linear regression," in *An Introduction to Audio Content Analysis: Music Information Retrieval Tasks and Applications*. Cham, Switzerland: Springer, 2023, pp. 409–410.
[31] R. Genuer and J.-M. Poggi, *Random Forests*. Cham, Switzerland: Springer, 2020.
[32] C. Bentéjac, A. Csörgő, and G. Martínez-Muñoz, "A comparative analysis of gradient boosting algorithms," *Artif. Intell. Rev.*, vol. 54, no. 3, pp. 1937–1967, Mar. 2021.
[33] D. A. Pisner and D. M. Schnyer, "Support vector machine," in *Machine Learning*. Amsterdam, The Netherlands: Elsevier, 2020, pp. 101–121.
[34] M. Sabzekar and S. M. H. Hasheminejad, "Robust regression using support vector regressions," *Chaos, Solitons Fractals*, vol. 144, Mar. 2021, Art. no. 110738.
[35] K. Taunk, S. De, S. Verma, and A. Swetapadma, "A brief review of nearest neighbor algorithm for learning and classification," in *Proc. Int. Conf. Intell. Comput. Control Syst. (ICCS)*, May 2019, pp. 1255–1260.
[36] L. Connelly, "Logistic regression," *Medsurg Nursing*, vol. 29, no. 5, pp. 1–10, 2020.
[37] R. D. King, O. I. Orhobor, and C. C. Taylor, "Cross-validation is safe to use," *Nature Mach. Intell.*, vol. 3, no. 4, p. 276, Apr. 2021.
[38] A. Banga, R. Ahuja, and S. C. Sharma, "Performance analysis of regression algorithms and feature selection techniques to predict $PM_{2.5}$ in smart cities," *Int. J. Syst. Assurance Eng. Manage.*, vol. 14, no. S3, pp. 732–745, Jul. 2023.
[39] G. Naidu, T. Zuva, and E. M. Sibanda, "A review of evaluation metrics in machine learning algorithms," in *Proc. Comput. Sci. On-Line Conf.* Cham, Switzerland: Springer, Jan. 2023, pp. 15–25.
[40] *HDFS*. Accessed: Aug. 12, 2024. [Online]. Available: https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs
[41] *Yarn*. Accessed: Aug. 12, 2024. [Online]. Available: https://yarnpkg.com/
[42] T. Kurita, "Principal component analysis (PCA)," in *Computer Vision: A Reference Guide*. Cham, Switzerland: Springer, 2020, pp. 1–4.



ELIAS DRITSAS received the Diploma, M.Sc., and Ph.D. degrees in computer science and informatics from the Department of Computer Engineering and Informatics, University of Patras, Greece, and the M.B.A. degree from the University of Derby, U.K. He participated in many development projects funded by the EC and the Greek Secretariat of Research and Technology. He is the author and co-author of publications in the area of machine learning and data analysis. His research interests include AI and machine learning, big data, databases, data mining, human–computer interaction, cloud computing, security, and trust. He is a member of the Technical Chamber of Greece and the Association for Computing Machinery. He serves as a regular reviewer for several technical journals and conferences.



MARIA TRIGKA received the Diploma degree from the Department of Electrical and Computer Engineering and the master's and Ph.D. degrees in signal processing for wireless communications from the Department of Computer Engineering and Informatics, University of Patras, Greece. As a Ph.D. candidate, she was a Research Scholar under the project "Strengthening Human Research Potential through Doctoral Research" co-financed by Greece-State Scholarships Foundation (IKY) and the European Union. Also, she has been involved in research projects co-financed by the European Union and Greek national funds through the Operational Program Competitiveness, Entrepreneurship and Innovation, under the call RESEARCH–CREATE–INNOVATE that emphasized artificial intelligence and machine learning for developing health and cultural services for hard of hearing. She is the author and co-author of publications in the area of signal processing, wireless communications, and machine learning.